君正®

Zeratul 媒体内存池使用说明

Date: July.2020

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®
Zeratul 媒体内存池使用说明
Copyright © Ingenic Semiconductor Co. Ltd 2019. All rights reserved.

Release history

Date	Revision	Change
July.2020	1.0	First release
Jan.2021	1.1	增加低功耗使用注意事项

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

北京君正集成电路股份有限公司

地址: 北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:(86-10)56345000
传真:(86-10)56345001
Http: //www.ingenic.c

目录

1 媒体内存池 1
1.1 RMEM 媒体内存池简介 1
1.2 MEMPOOL 概念 1
1.3 内存使用方法及相关 API 简介 2
1.4 Mempool size 大小计算 4
1.5 Mempool 内存使用情况查看 4
1.6 低功耗平台使用注意 5

Copyright® 2005-2020 Ingenic Semiconductor Co., Ltd. All rights reserved.

1 媒体内存池

1.1 RMEM 媒体内存池简介

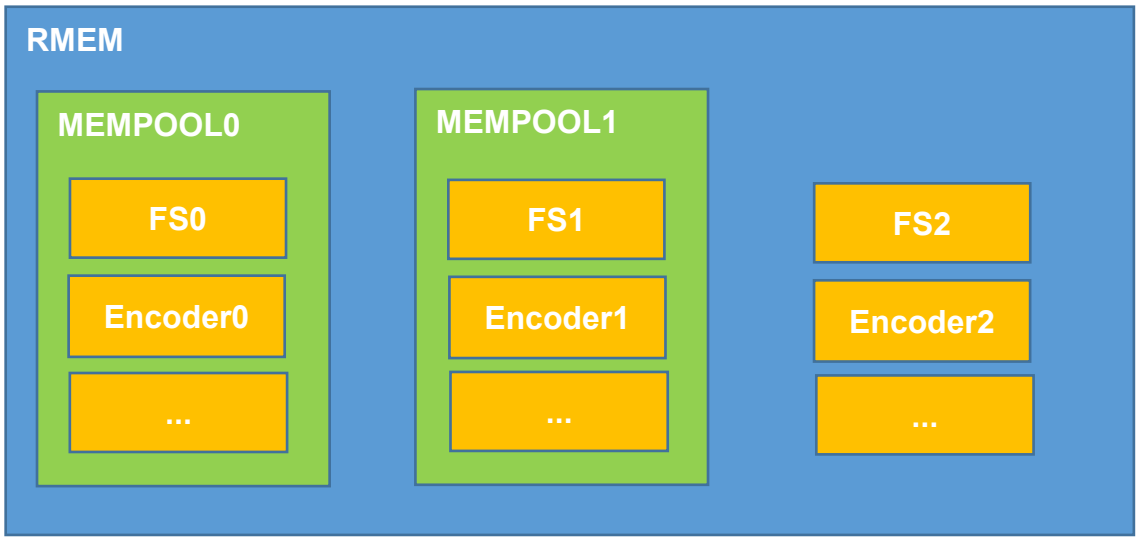
芯片真实的物理内存会被划分为两块，由 cmdline 来划分，如：mem=64M@0x0 rmem=64M@0x4000000，其中 mem 用于 linux 系统内存，rmem 用于媒体内存。

媒体内存池主要向媒体业务提供大块物理连续内存管理功能,负责内存的分配和回收,充分发挥内存缓存池的作用,让物理内存资源在各个媒体处理模块中合理使用。视频内存池建立在原保留内存 RMEM（不由内核建立页表）的基础上进行大块内存管理，每申请一个内存池都是物理连续的内存，之后在该内存池上申请内存同样是申请物理连续的内存。若使用内存池，必须在系统初始化之前配置内存池大小，根据业务的不同，内存池申请的大小和数量不同。

1.2 MEMPOOL 概念

rmem 作为媒体内存的整体，而 mempool 作为内存池的最小单位，mempool 也是在 rmem 中进行申请和释放，作为 rmem 的局部内存管理单元。

关系如下图：



使用 mempool 主要的作用是将某一些具有关联性的一些内存申请放到同一个内存池，如 fs0、osd0、encoder0 等放到一个独立的内存池中，当这一路流需要切换分辨率的时候，这些资源是需要同时释放，再重新分配的，那么在这个过程中，不会影响到 mempool1 和其他内存块，这样可以防止内存碎片的产生。

mempool 的设置也是比较灵活，可以随意绑定。绑定到 mempool 后，模块内存申请从绑定的 mempool 中申请，不绑定到 mempool 中的内存默认从 rmem 中申请。

Copyright® 2005-2020 Ingenic Semiconductor Co., Ltd. All rights reserved.

1.3 内存使用方法及相关 API 简介

使用方法参考如下：

```
   // 初始化 mempool，从 rmem 中申请 pool
   IMP_System_MemPoolRequest(0, 12 *(1 << 20), "pool0");
   IMP_System_MemPoolRequest(1, 12 *(1 << 20), "pool1");

   // 将 framesource 通道 0 绑定到 pool0 上，将 framesource 通道 1 绑定到 pool1 上，
这个绑定可以根据需求来进行，如果不绑定 mempool，则默认从 rmem 中申请。
   IMP_FrameSource_SetPool(1, 1);
   IMP_FrameSource_SetPool(0, 0);

   // 将编码通道 0 绑定到 pool0 上，将编码通道 1 绑定到 pool1 上
   IMP_Encoder_SetPool(0, 0);
   IMP_Encoder_SetPool(1, 1);
   ...
}
```

相关 API 说明：

```
/**
 * @brief IMP_System_MemPoolRequest(int poolId, size_t size, const char
*name);
 * 在 Rmem 上申请 mempool
 * @param[in] poolId      申请的 poolID.
 * @param[in] size        申请大小.
 * @param[in] name        pool 名称.
 * @retval    0           成功.
 * @retval    非 0        失败.
 * @attention 无.
 */
int IMP_System_MemPoolRequest(int poolId, size_t size, const char *name);

/**
 * @brief IMP_FrameSource_SetPool(int chnNum, int poolID);
 * 绑定 chnnel 到内存池中，即 FrameSource 申请 mem 从 pool 申请.
 * @param[in] chnNum     通道编号.
 * @param[in] poolID     内存池编号.
 * @retval 0             成功.
 * @retval 非 0          失败.
 * @attention 无.
 */
int IMP_FrameSource_SetPool(int chnNum, int poolID);
```


Copyright® 2005-2020 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
/**
 * @brief IMP_FrameSource_GetPool(int chnNum);
 * 通过 channel ID 获取 poolID.
 * @param[in] chnNum       通道编号.
 * @retval  >=0 && < 32    成功.
 * @retval  <0             失败.
 * @attention 无.
 */
int IMP_FrameSource_GetPool(int chnNum);

/**
 * @brief int IMP_Encoder_SetPool(int chnNum, int poolID);
 * 绑定 chnnel 到内存池中，即 Encoder 申请 mem 从 pool 申请.
 * @param[in] chnNum     通道编号.
 * @param[in] poolID     内存池编号.
 * @retval 0             成功.
 * @retval 非 0              失败.
 * @attention 无.
 */
int IMP_Encoder_SetPool(int chnNum, int poolID);

/**
 * @brief IMP_Encoder_GetPool(int chnNum);
 * 通过 channel ID 获取 poolID.
 * @param[in] chnNum       通道编号.
 * @retval  >=0 && < 32    成功.
 * @retval  <0              失败.
 * @attention 无.
 */
int IMP_Encoder_GetPool(int chnNum);
```


Copyright® 2005-2020 Ingenic Semiconductor Co., Ltd. All rights reserved.

1.4 Mempool size 大小计算

在 rmem_calculator.xls 标签 mempool 计算器，按照使用说明会自动计算出 pool 大小

使用说明:
1.设置Sensor、主路、次路、第三路分辨率，图像格式和nrVbs大小;
2.设置内存大小
3.设置编码分辨率(需要注意编码尺寸和通道分辨率不同的情形）;
4.设置编码类型，在Codec一栏选择HEVC即265编码、选择AVC即264编码，不同编码类型内存使用不一样;
5、设置是否是能通道:En，是否是能jpeg编码:EnJpeg;
6、其余选项设置可以按照默认设置进行
最后会自动算出mempool的大小，ps：mempool是依赖于rmem的大小，因此确定rmem大小

设置分辨率			图像格式	nrvbs
Sensor分辨率	1920	1080		
主路	1920	1080	NV12	2
次路	640	360	NV12	2
第三路	1280	720	BGR0	2

PoolId	En	width	height	depth	nrvbs	ncu	编码width	编码height	Codec	eGopMode	uGopLength	bEnableLtr	uNumB	stream_buffer	FORCE_REC	OSD	EnJpeg	总和
1	Yes	1920	1080	1.5	2	0	1920	1080	HEVC	DEFAULT	50	FALSE	0	2	FALSE	0	No	14553088
2	Yes	640	360	1.5	2	0	640	360	HEVC	DEFAULT	50	FALSE	0	2	FALSE	0	No	1794048
3	No	1280	720	4	2	0	1280	720	HEVC	DEFAULT	50	FALSE	0	2	FALSE	0	No	0

1.5 Mempool 内存使用情况查看

应用程序运行之后，在终端执行 cat /tmp/mempool-[your pool ID]

```
================ continuous dump info End ============
[root@Ingenic-uc1_1:tmp]# cat /tmp/mempool-pool0

================ continuous dump info ============

name = [pool0] begin addr = 0x7373d000 size = 2048 used = 1
name = [unknown] begin addr = 0x7373d800 size = 25600 used = 1
name = [ep1] begin addr = 0x73743c00 size = 5376 used = 1
name = [ep3] begin addr = 0x73745100 size = 5376 used = 1
name = [ep3] begin addr = 0x73746600 size = 5376 used = 1
name = [ep3] begin addr = 0x73747b00 size = 9984 used = 1
name = [cmdlist] begin addr = 0x7374a200 size = 3277312 used = 1
name = [ref + rec + mv] begin addr = 0x73a6a400 size = 3277312 used = 1
name = [ref + rec + mv] begin addr = 0x73d8a600 size = 42496 used = 1
name = [interm] begin addr = 0x73d94c00 size = 949248 used = 1
name = [stream] begin addr = 0x73e7c800 size = 949248 used = 1
name = [stream] begin addr = 0x73f64400 size = 6266880 used = 1
name = [VBMPool0] begin addr = 0x7455e400 size = 912384 used = 0
Mem Total Size: 15728640
Mem Used  Size: 14816256
Mem Free  Size: 912384
Mem Usage Rate: 94%

================ continuous dump info End ============
[root@Ingenic-uc1_1:tmp]#
```


Copyright® 2005-2020 Ingenic Semiconductor Co., Ltd. All rights reserved.

注意：

IMP_FrameSource_SetPool 绑定 channel 与 mempool 一定要在 channel 创建之前调用

1.6 低功耗平台使用注意

低功耗产品，会在内核缓存一定数量的帧数据，这些内存由内核申请，最终和上层应用共享这块内存。

由于快起数据主要用于视频录像，所以内核只对主码流进行缓存。

Mempool 主要针对上层 SDK 而言，由于内核已经申请过主码流 framesource 的 yuv buffer 了，如果我们再把主码流 framesource 再次绑定到 mempool 中，这样会导致 framesource 模块在 mempool 中再次申请内存，重复申请造成内存浪费。

所以实际使用中主码流 framesource 不需要绑定到 mempool 中。这样，主码流 framesource 会继续使用内核共享的这块内存。同时内核申请的这块内存也是做过处理的，频繁切换分辨率不会导致内存碎片产生。

Copyright® 2005-2020 Ingenic Semiconductor Co., Ltd. All rights reserved.